Multicore & GPU Programming : An Integrated Approach

Shared-memory programming : Threads

By G. Barlas

Objectives

- Learn what threads are and how you can create them.
- Learn how to initialize threads in order to perform a desired task.
- Learn how to terminate a multi-threaded program using different techniques.
- Understand problems associated with having threads access shared resources, like race-conditions and deadlocks.
- Learn what semaphores and monitors are and how you can use them in your programs.
- Become familiar with classical synchronization problems and their solutions.
- Learn how to create and use a pool of threads.
- Learn effective debugging techniques for multi-threaded programs.

Introduction

- **Thread** : execution path, a sequence of instructions, that is managed separately by the operating system scheduler as a unit. There can be multiple threads per process
- Threads are good for:
  - Improving performance
  - Background tasks
  - Asynchronous processing
  - Improving program structure

Thread creation

- Traditionally, threads are handled by third-party libraries, such as:
  - **pThreads**
  - **winThreads** : a Windows-only C++ based library
  - **Qt threads** : part of the Qt cross-platform C++ toolkit
- C++11 standard includes a build-in thread library.
- The facilities provided by the standard have improved over subsequent iterations, including C++20.

Threads in C++11

```
#include <thread>

using namespace std;
void f()
{
  cout <<"Hello from thread " << this_thread::get_id() << "\n";
  this_thread::sleep_for(1s);
}

int main(int argc, char **argv)
{
  thread t(f);
  cout << "Main thread waiting...\n";
  t.join();
  return 0;
}
```

Compiling thread code in C++11

- The **`join`** blocking call prevents the program from terminating, also killing the child thread.
- All threads get an implicit identifier, that can be queried via the `this_thread` namespace.
- To compile and run:

```
$ g++ hello.cpp -lpthread -o hello
$ ./hello
Main thread waiting...
Hello from thread 139981974968064
```

- The compilation command implies the use of **pThreads** as the underlying implementation basis.

Passing parameters to threads

- Using **global variables** is one possibility:
  - By assigning an explicit ID to each thread (not the one returned by **`get_id()`**), we can have each thread "look" into its own dedicated data in specific arrays.
  - Generally not recommended.
- Using a **list of parameters** in the function called by the thread constructor.
- Using a **functor**: an instance of a class that implements the **`operator()`** method.
  - Most elegant solution

Example: finding the average using a list of parameters

```
void partialSum(vector<double>::iterator start, vector<double>::iterator end,
double *res) {
    *res=0;
    for(auto i=start; i<end; i++)
        *res += *i; }
...
  int step=(int)ceil(N*1.0/numThr);
  double res[numThr];
  vector<double> data(N);
  thread *thr[numThr];
  vector<double>::iterator localStart = data.begin(), localEnd;
  for(int i=0;i<numThr;i++)  {
      localEnd=localStart+step;
      if(i==numThr-1) localEnd=data.end();
      thr[i] = new thread(partialSum, localStart, localEnd, res+i);
      localStart += step;
  }
```

Example: finding the average using a list of parameters (cont.)

```
double total=0;
for(int i=0;i<numThr;i++)
{
    thr[i]->join();
    delete thr[i];
    total += res[i];
}
cout << "Average is : " << total/N << endl;
```

- Important note: all parameters in the **`thread`** constructor, are passed by value.
- Passing by reference requires the use of the **`ref()`** and **`cref()`** functions.

Example: finding the average using a functor

```
struct PartialSumFunctor
{
  vector < double >::iterator start;
  vector < double >::iterator end;
  double res = 0;
  void operator() ();
};
void PartialSumFunctor::operator() ()
{
  for (auto i = start; i < end; i++)
    res += *i;
}
```

Example: finding the average using a functor (cont.)

```
int main (int argc, char **argv)
{
  ...
  unique_ptr < thread > thr[numThr - 1];
  PartialSumFunctor f[numThr];
  for (int i = 0; i < numThr - 1; i++)
    {
      localEnd = localStart + step;
      f[i].start = localStart;
      f[i].end = localEnd;
      thr[i] = make_unique < thread > (ref (f[i]));
      localStart += step;
    }
```

Array of smart pointers. Each one points to an instance of thread.

Array of functors. Notice the mismatch in array size.

Thread starts as soon as it is constructed.

Any side-effects if `ref()` were not used?

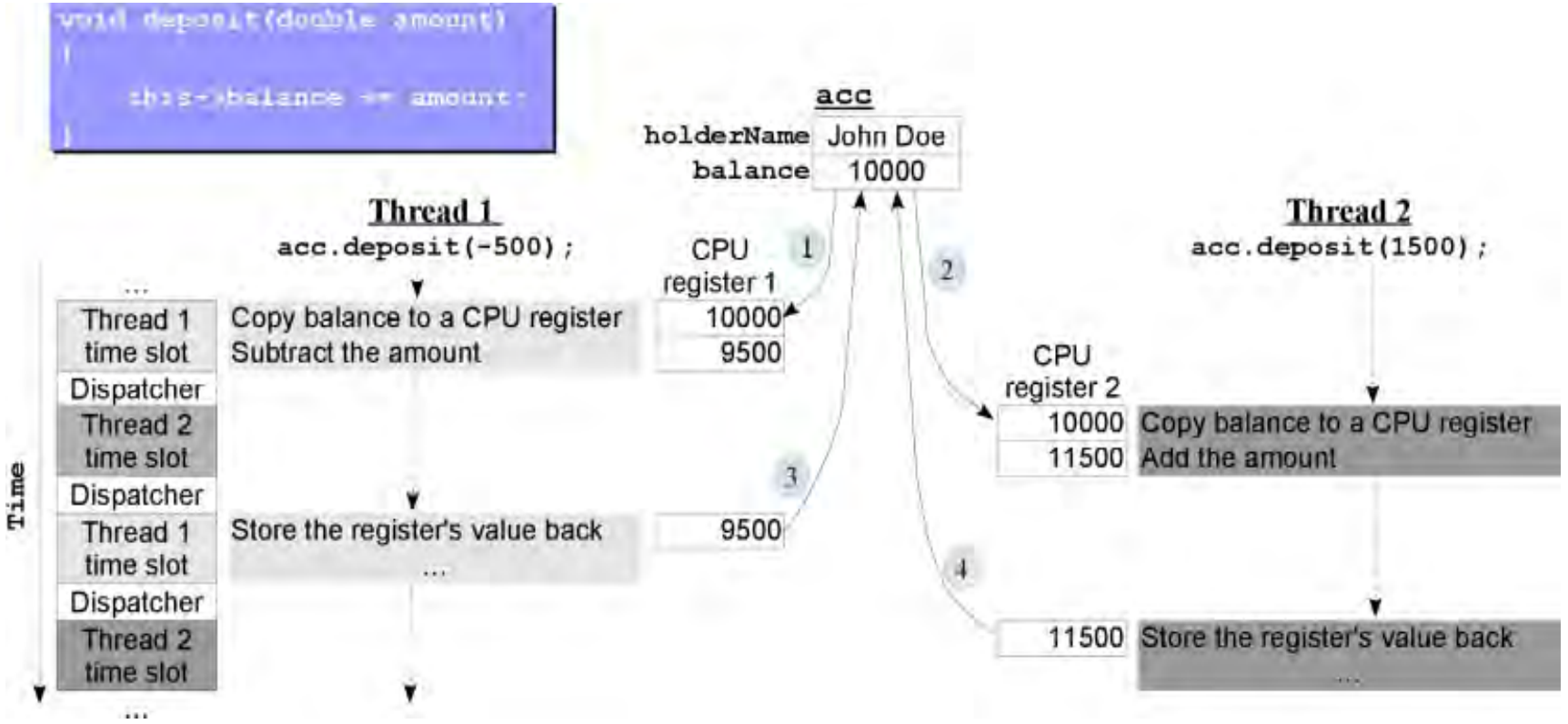
Example: finding the average using a functor (cont.)

```
f[numThr - 1].start = localStart;
f[numThr - 1].end = data.end ();
f[numThr - 1] ();  // main thread processes workload part
double total = f[numThr - 1].res;


for (int i = 0; i < numThr - 1; i++)
  {
    thr[i]->join ();
    total += f[i].res;
  }
```

No thread here.

Data sharing between threads

- A **data race** occurs when two or more threads access the same memory location and at least one of those accesses is a write.
- **Race condition**: an abnormal program behavior caused by dependence of the result on the relative timing of events in the program.



Data races VS race conditions

- A (pseudocode) method with no data races but with a race condition:

```
void deposit(double amount)
{
  double temp;
  atomic{ temp = this->balance; }
  temp += amount;
  atomic{ this->balance = temp; }
}
```

- A (pseudocode) method with a data race but no race condition:

```
void deposit(double amount)
{
  this-> modified = true;
  atomic{ this->balance += amount; }
}
```

- Race conditions are more difficult to discover and eliminate.
- Data races can be eliminated via **critical sections**.

A race condition in action

```
int main(int argc, char **argv)
{
  ...
  double res=0;
  ...
  for(int i=0;i<numThr;i++)
  {
      ...
      thr[i] = new thread(partialSum, localStart, localEnd, &res);
  }
...
  cout << "Average is : " << res/N << endl;
...
```

A race condition in action (2)

- The shared `res` variable will cause problems:

```
$ ./aver0  4 1000
Average is : 462.998
$ ./aver0  4 1000
Average is : 469.364
$ ./aver0  4 1000
Average is : 500.5
```

- The result in all the runs should have been

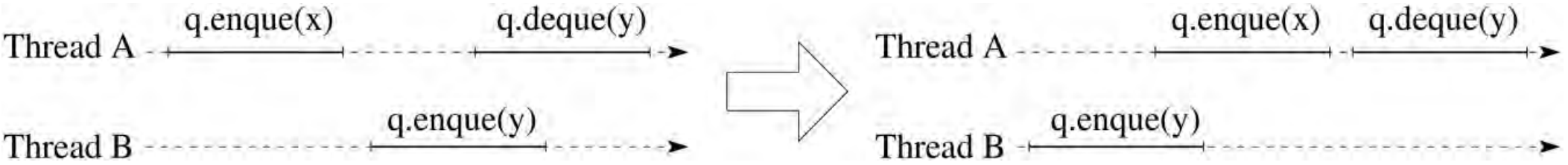
$$\frac{1000+1}{2} = 500.5$$

Consistency models

- The problem of how to operate on shared data is addressed by the so called ***consistency models***.
- A consistency model is a set of rules that dictate how operations on data should take effect.
- The **sequential consistency** model dictates that all events on shared objects should appear as if they were happening one-at-a-time. Events should appear to take place in program order as far as each thread is concerned.
- The problem is that the composition of software modules that satisfy sequential consistency is not necessary sequential consistent.

Sequential consistency example

- Time shifting of events allows their order to match a sequential order:

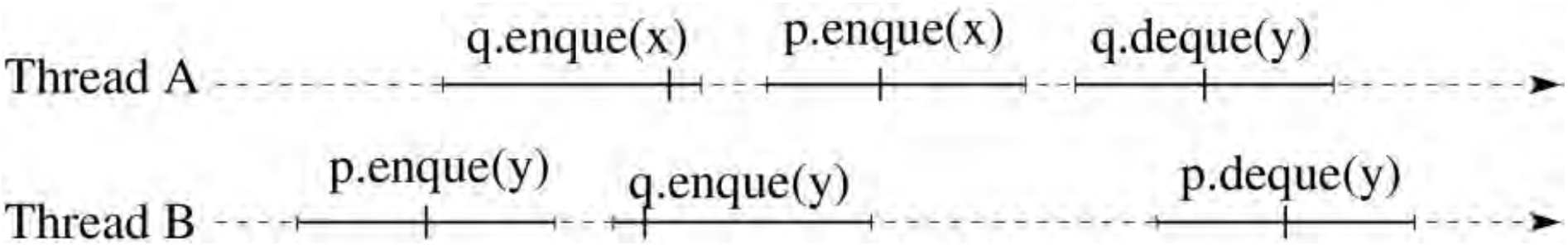


Linearizability

- In the **linearizability** consistency model, events should appear to happen in one-at-a-time order, and should appear to take place **instantaneously**.
- This is achieved by the introduction of so called **linearization points**. A linearization point is a time instance within the duration of a method call, where the effect of the call is considered to take effect.
- The introduction of the linearization points has two implications:
  - The methods can be totally ordered according to their linearization points.
  - Although locking is not mandated (and actually it should be avoided for performance reasons), locking down an object while a method is performed is the equivalent of having its effect take place instantaneously as far as everyone else is concerned.
- Every linearizable execution is sequentially consistent, but the reverse is not true.

Linearizability example

- Operating on two FIFO queues:



Semaphores

- A semaphore is a software abstraction proposed by the late Dutch computer scientist Edsger Dijkstra, that supports two atomic operations:
  - **P** for "try to decrease". Operation is blocking if the value of the semaphore is non-positive.
  - **V** for "increment"
- Semaphores are equipped with a queue for holding blocked threads.If the queue is a FIFO queue the semaphore is called *strong* (or *weak* otherwise)
- A **binary semaphore** can have only two states. Blocking occurs if the value is false.
- A mutex (**mut**ual **ex**clusion) is a restricted variant of the binary semaphore that can only be unlocked by the thread that locked it.

Semaphore classes in C++

- C++11 only supports a mutex class:

```
#include<mutex>
std::mutex l;
l.lock();
l.unlock();
```

- C++20 added support for a counting semaphore:

```
#include<semaphore>
std::counting_semaphore<10> resCount;
resCount.acquire();
resCount.release();
```

"10" is the maximum value.

- Because of incomplete C++20 support in current compilers, in this chapter we use a user-supplied class with similar functionality:

```
#include"semaphore"
semaphore resCount(10);
```

"10" is the initial value.

- A binary semaphore is just an instance of:

```
std::counting_semaphore<1> lock;
```

Using a semaphore

- A semaphore can be utilized in three distinct ways:
  - As a **lock**. Suitable semaphore type : **binary** or mutex.
  - As a **resource counter**. Suitable semaphore type : **general**.
  - As a **signaling mechanism**. Suitable semaphore type : **binary** or **general** depending on the application.

Semaphore use patterns

	Use	*Sem. Initialization*	*Thread1*	*Thread2*	*Thread Relationship*

S1	Lock	`mutex l;`	`l.lock();`
			`...`
			`l.unlock();`
				`l.lock();`
				`...`
				`l.unlock();`
					Threads compete for acquiring the lock to a shared resource.

S2	Resource Counter Scenario 1	`semaphore s;`	`s.release();`	`s.acquire();`	One thread produces resources (Thread 1) and another consumes them (Thread 2). Initially there are no available ones.

S3	Resource Counter Scenario 2	`semaphore s(N);`	`s.acquire();`
			`...`
			`s.release();`
				`s.acquire();`
				`...`
				`s.release();`
					Thread compete for acquiring resources from a pool of N available ones.

S4	Signaling Mechanism	`semaphore s;`	`s.release();`	`s.acquire();`	One thread (Thread 2) waits for a signal from the other (Thread 1).

Common problem patterns: producers-consumers

- A common buffer for communicating instances of a Resource class between threads, can be established as:

```
const int BUFFSIZE=100;
class Resource {...};

Resource buffer[BUFFSIZE];
int in=0, out=0;

semaphore slotsAvail(BUFFSIZE);   // counts how many buffer slots are free
semaphore resAvail(0);            // counts how many buffer slots are taken
mutex l1, l2;                     // by default initialized open
```

Producers code

```
class Producer {
public:
  void operator() ()  {
     while(1) {
        Resource item = produce();
        slotsAvail.acquire() ;   // wait for an empty slot in the buffer
        l1.lock();
        buffer[in] = item;       // store the item
        in = (in+1) % BUFFSIZE;  // update the in index safely
        l1.unlock();
        resAvail.release();      // signal resource availability
     }
  }
};
```

Consumers code

```
class Consumer {
public:
  void operator() ()  {
     while(1) {
        resAvail.acquire() ;     // wait for an available item
        l2.lock();
        Resource item = buffer[out];// take the item out
        out = (out+1) % BUFFSIZE;// update the out index
        l2.unlock();
        slotsAvail.release();    // signal for a new empty slot
        consume(item);
      }
  }
};
```

Producers-consumers semaphore setup

Consumers \ Producers	1	N
1	2 counting semaphores for signaling	2 counting semaphores and 1 binary semaphore for the producers
M	2 counting semaphores and 1 binary semaphore for the consumers	2 counting semaphores and 2 binary semaphores

Termination problem

- How can we stop the threads upon program termination in an orchestrated manner?
- If the main/parent thread exits, all children are terminated also by the O.S. But this is not the proper course of action.
- Two possibilities:
  - Shared data item
  - Messages

Termination using a shared variable

- Changes to the shared variable should be detected as soon as possible.
- There are two possibilities:
  - The number is iterations is known a priori.
  - The number is iterations is determined at run-time.
- The first case can be accommodated by a counting semaphore.
- The second case requires the declaration of a **`volatile`** shared variable.

Example : terminating producers-consumers – case I

- Assuming that producers/consumers are supposed to process a fixed number of items:

```
template<typename T>
class Producer {
```

```
    static semaphore * numProducts;
```

```
template<typename T>
void Producer<T>::operator()() {
    while (numProducts->try_acquire()) {
        T item = (*produce)();
        slotsAvail->acquire(); // wait for an empty slot in the buffer
        l1.lock();
        buffer[in] = item; // store the item
        in = (in + 1) % BUFFSIZE; // update the in index safely
        l1.unlock();
        resAvail->release(); // signal resource availability
    }
}
```

Example : terminating producers-consumers - case II

- Assuming that producers/consumers are supposed to continue until a condition is met:

```
template<typename T>
class Producer {
private:
    . . .
    static volatile bool *exitFlag;
```

```
template<typename T>
void Producer<T>::operator()() {
    while (*exitFlag == false) {
        T item = (*produce)();
        slotsAvail->acquire(); // wait for an empty slot in the buffer

        if (*exitFlag) return; // stop immediately on termination

        l1.lock();
        buffer[in] = item; // store the item
        in = (in + 1) % BUFFSIZE; // update the in index safely
        l1.unlock();
        resAvail->release(); // signal resource availability
    }
}
```

Example : terminating producers-consumers - case II (cont.)

- A consumer thread is suppose to trigger the termination:

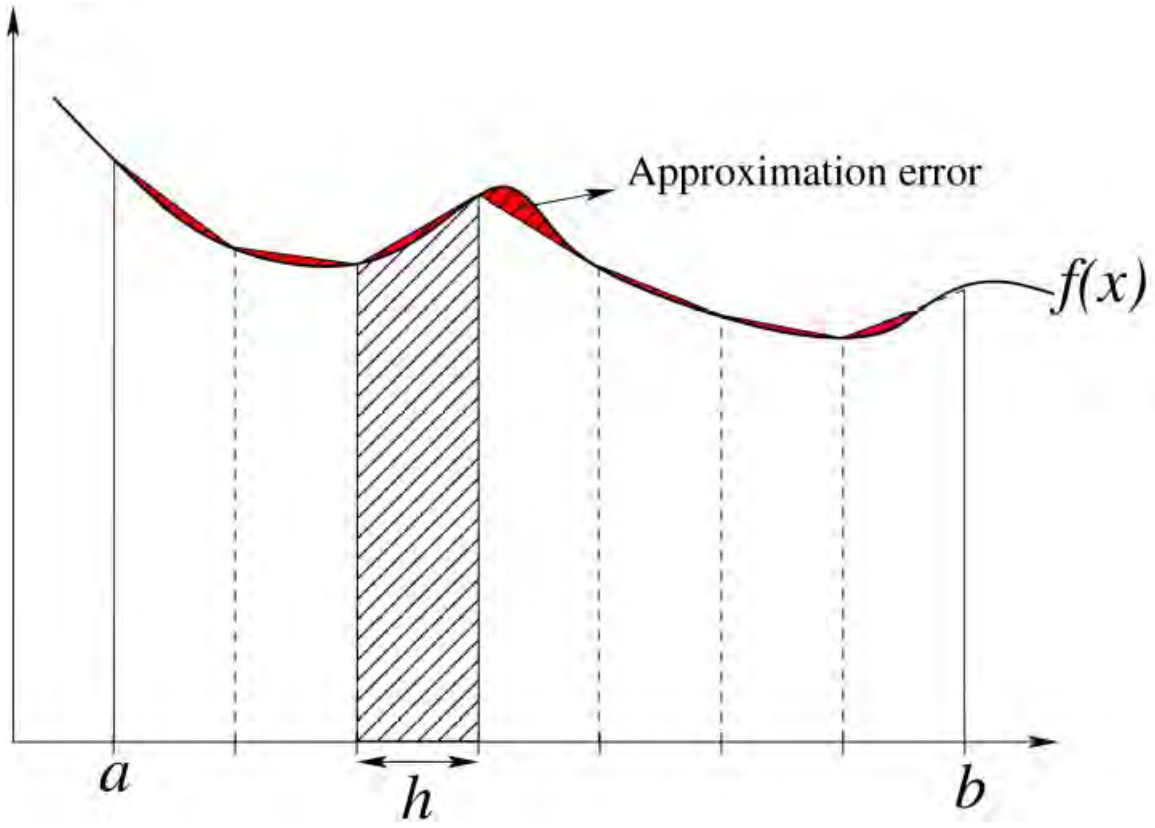
```
template<typename T> void Consumer<T>::operator()() {
    while (*exitFlag == false) {
        resAvail->acquire(); // wait for an available item

        if (*exitFlag) return; // stop immediately on termination

        l2.lock();
        T item = buffer[out]; // take the item out
        out = (out + 1) % BUFFSIZE; // update the out index
        l2.unlock();
        slotsAvail->release(); // signal for a new empty slot

        if ((*consume)(item)) break; // time to stop?
    }

    // only the thread initially detecting termination reaches here
    *exitFlag = true;
    resAvail->release(numConsumers - 1);
    slotsAvail->release(numProducers);
}
```

Termination using messages

- Example: trapezoid rule integration using a single producer – multiple consumers formulation.
- Background:

$$\sum_{i=0}^{n-1} h \cdot \frac{f(x_i) + f(x_{i+1})}{2} =$$

$$h\left(\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(x_i)\right)$$



Termination using messages (2)

- Integration interval is split into smaller slices, represented by structure:

```
typedef struct Slice {
    double start;
    double end;
    int divisions;
} Slice;
```

- Each consumer, calculates the area of a slice it reads from the buffer.
- Until `divisions` are set to 0.

Trapezoid "consumer"

```
void IntegrCalc::operator()() {
    while (1) {
        resAvail->acquire(); // wait for an available item
        l2.lock();
        int tmpOut = out;
        out = (out + 1) % BUFFSIZE; // update the out index
        l2.unlock();

        // take the item out
        double st = buffer[tmpOut].start;
        double en = buffer[tmpOut].end;
        int div = buffer[tmpOut].divisions;

        slotsAvail->release(); // signal for a new empty slot

        if (div == 0) break; // exit
```

Trapezoid "consumer" (2)

```
        //calculate area
        double localRes = 0;
        double step = (en - st) / div;
        double x;
        x = st;
        localRes = func(st) + func(en);
        localRes /= 2;
        for(int i=1; i< div; i++)   {
            x += step;
            localRes += func(x);
          }
        localRes *= step;

        // add it to result
        resLock.lock();
        *result += localRes;
        resLock.unlock();
    }
}
```

Trapezoid "producer" : parent thread

Work generation

```
double divLen = (UPPERLIMIT - LOWERLIMIT) / J;
double st, end = LOWERLIMIT;
for (int i = 0; i < J; i++) {
    st = end;
    end += divLen;
    if (i == J - 1) end = UPPERLIMIT;

    buffSlots.acquire();
    buffer[in].start = st;
    buffer[in].end = end;
    buffer[in].divisions = 1000;
    in = (in + 1) % BUFFSIZE;
    avail.release();
}
```

Termination

```
// put termination sentinels in buffer
for (int i = 0; i < N; i++) {
    buffSlots.acquire();
    buffer[in].divisions = 0;
    in = (in + 1) % BUFFSIZE;
    avail.release();
}
```

Atomic data types

- C++11 and subsequent versions of the standard have introduced an `atomic` class template and a number of specializations.
- The atomic data types offer data race-free access to threads without needing to resort to explicit locking.
- Examples:

Alias	Specialization
std::atomic_bool	std::atomic<bool>
std::atomic_char	std::atomic<char>
std::atomic_short	std::atomic<short>
std::atomic_int	std::atomic<int>
std::atomic_long	std::atomic<long>
std::atomic_llong	std::atomic<long long>
std::atomic_char16_t	std::atomic<char16_t>
std::atomic_char32_t	std::atomic<char32_t>
std::atomic_int8_t	std::atomic<std::int8_t>
std::atomic_int16_t	std::atomic<std::int16_t>
std::atomic_int32_t	std::atomic<std::int32_t>
std::atomic_int64_t	std::atomic<std::int64_t>
std::atomic_intptr_t	std::atomic<std::intptr_t>
std::atomic_size_t	std::atomic<std::size_t>
std::atomic_ptrdiff_t	std::atomic<std::ptrdiff_t>
std::atomic_intmax_t	std::atomic<std::intmax_t>

Atomic data types methods

- One cannot pass any data type as the atomic class template argument.
- However, for the support data types, the atomic equivalents provide the following methods:
  - `store`/`load` : saves/retrieves the value of the atomic object.
  - `fetch_add`/`fetch_sub` : adds/subtracts the argument from the stored value. The method returns the previous value.
  - `fetch_and`/`fetch_or`/`fetch_xor` : performs bitwise AND/OR/XOR between the argument and the value of the atomic object. The previous value is returned.
  - `exchange` : replaces the value of the atomic object and returns the previous value.
  - `compare_exchange_weak`/`compare_exchange_strong` : compares the value of the atomic object with the argument and performs an exchange operation if they are equal or a load operation otherwise.
  - `is_lock_free` : returns true if the atomic object is lock-free, i.e., a lock is not used to implement the aforementioned methods.
- A number of atomic augmented assignment and increment/decrement operators are also provided: ++, - -, +=, -=, &=, etc., but these are type specific.

Atomic data based mutual exclusion

- One can create a custom lock by utilizing the atomic types.
- Example:

```
atomic<bool> lock; // should be accessible from all concerned threads

// entry section
while(true)
{
   bool tmp = false;
   if(lock.compare_exchange_weak(tmp, true)
      break;
   this_thread::yield();
}

// critical section
. . .

// exit section
lock.store(false);
```

Memory ordering

- Memory ordering is concerned with inter-thread coordination, i.e., how memory operations performed in one thread are perceived by other threads.
- Memory ordering settings specify how consistency between copies of the same data existing across different cache hierarchies and register files, is maintained.
- The C++11 standard established the following memory orderings, that are optional parameters to atomic type operations:
  - `memory_order_relaxed` : there are no synchronization or ordering constraints imposed on other threads' reads or writes.
  - `memory_order_acquire` : used in a load operation, to make sure that all writes in other threads to the concerned atomic variable, are visible in the current thread.
  - `memory_order_release` : used in a store operation to ensure that the changes from the current thread become visible in other threads that acquire/read the same atomic variable.
  - `memory_order_acq_rel` : used in read-modify-write operation, where both an acquire operation is performed for the read part, and a release operation is performed for the write part. Hence changes from other threads become visible before the read, and the change from the current thread becomes visible to other threads after the write.
  - `memory_order_seq_cst` : short for "sequentially consistent," i.e., a load operation performs an acquire and a write operation performs a release. This is the default setting for ensuring program correctness.

Monitors

- A monitor is an object that encapsulates thread coordination logic. Its methods are mutually exclusive for the threads using it, and all its data members are private.
- In order to be able to block a thread, or signal a thread to continue (both essential functionalities for establishing critical sections), monitors provide a mechanism called a **condition variable** (aka wait condition).
- A condition variable, which is typically accessed only inside a monitor, has a queue and supports two operations (actual names depend on the implementation):
  - `wait` : if a thread executes the wait operation of a condition variable, it is blocked and placed in the condition variable's queue.
  - `signal` : if a thread executes the signal operation of a condition variable, a thread is popped from the condition variable's queue and it becomes ready. If the queue is empty, the signal is ignored.

Monitors, cont.

- Each instance of a monitor must be equipped with a mutex that is locked on entry to a method, and unlocked upon exit.
- Monitor methods are made up of three parts:
  - **Entry** (optional) : checking if the conditions are right for a thread to proceed.
  - **Mid section** : manipulating the state of the monitor to the desired effect, plus performing any operations that require mutual exclusion.
  - **Exit** (optional): signaling to other threads that it is their turn to enter or continue execution in the monitor.

Monitors, example

- Pseudocode of a bank account class:

```
class AccountMonitor
{
  private:
    Condition insufficientFunds;
    Mutex m;
    double balance;
  public:
    void withdraw(double s) {
      m.lock();
      if(balance < s)
        {
          m.unlock();
          insufficientFunds.wait();
          m.lock();
        }
      balance -= s;
      m.unlock();
    }
    void deposit(double s) {
      m.lock();
      balance += s;
      insufficientFunds.signal();
      m.unlock();
    }
};
```

Condition variables

- Most implementations follow the the Lampson-Redell specification, which does not mandate when a thread that woke-up from a signal operation, continues inside a monitor.
- This means that conditions must be checked again before a thread is allowed to proceed.
  - This means the `if` in the previous example, must be converted to a `while`.
- The benefit is that additional functionality can be afforded, such as timed-waits or waking all the waiting threads.

Condition variables in C++11

- C++11 provides a `condition_variable` class that supports the following methods:
  - `wait` : as described previously. A reference to the monitor-locking mutex (or a `unique_lock` instance) is required for releasing the monitor when the thread is forced to wait.
  - `wait_for` : forces a thread to block for a specified amount of time, or until it receives a signal.
  - `wait_until` : similar to wait_for, but the time-out is triggered when a specific time point is reached.
  - `notify_one` : notifies one of the blocked threads.
  - `notify_all` : notifies all blocked threads. They will all run, one-by-one, inside the monitor.

Locking the monitor

- In practice the mutex used to lock a monitor is not explicitly handled.
- Helper classes automate the locking and unlocking actions:
  - `unique_lock`
  - `lock_guard`
- Both take a reference to a mutex as a parameter to their constructor, which they proceed to lock.
- Upon their destruction, the mutex is unlocked, allowing other threads to use the monitor.
- The difference between the two is that the `unique_lock` permits the unlocking and locking of the mutex multiple times, as per the needs of the `wait` method.
- The `lock_guard` is employed in methods where `wait` is not called.

Monitor skeleton

```
class Monitor
{
  private:
    mutex l;      // monitor lock
    condition_variable cv, . . .; // one or more condition variables
  public:
    void foo1();
    void foo2();
    . . .
};
void foo1()
{
  unique_lock< mutex > ul( l );
  . . .
  cv.wait( ul, [](){ return someCondition; } );
  . . .
}
void foo2()
{
  lock_guard< mutex > guard( l );
  . . .
  cv.notify_one());
  . . .
}
```

Lambda function based variant of a `while-wait` loop

Account example in working code

```
#include <condition_variable>
class AccountMonitor
{
  private:
    condition_variable insufficientFunds;
    mutex m;
    double balance=0;
  public:
    void withdraw(double s) {
      unique_lock<mutex> ml(m);
      while(balance < s)
          insufficientFunds.wait(ml);
      balance -= s;
     }

    void deposit(double s) {
      lock_guard<mutex> ml(m);
      balance += s;
      insufficientFunds.notify_all();
     }
};
```

A check on whether s is positive or not is missing.

Alternative

```
insufficientFunds.wait(ml, [&](){
          return balance < s;   }
        );
```

Design #1 : Critical section inside monitor

```
class ConsoleMonitor
{
  private:
    mutex m;
  public:
    void printOut(string s) {
      lock_guard< mutex > g(m);
      cout << s;
    }
};
```

- What if the critical section is too time consuming?

Design #2 : Monitor controls entry to critical section

- Arrangement is similar to semaphore-based design.
- But monitors can check for very complicated conditions in an easy way.
- Example : producers-consumers implementation.
- This approach is sensible when buffer deposit/extraction is time-consuming (e.g. when it involves an deep copy of an object and not just the copy of a pointer).

Producers-consumers : buffer I/O exterior to the monitor

```
template<typename T>
class Monitor {
private:
    mutex l;
    condition_variable full, empty;
    queue<T *> emptySpotsQ;
    queue<T *> itemQ;
    T *buffer;
public:
    T* canPut();
    T* canGet();
    void donePutting(T *x);
    void doneGetting(T *x);
    Monitor(int n = BUFFSIZE);
    ~Monitor();
};
```

A single circular queue is no longer a viable option.

Can you guess why?

Producers-consumers : buffer I/O exterior to the monitor (2)

```
template<typename T>
Monitor<T>::Monitor(int n) {
    buffer = new T[n];
    for(int i=0;i<n;i++)
        emptySpotsQ.push(&buffer[i]);
}
//------------------
```

```
template<typename T>
T* Monitor<T>::canPut() {
    unique_lock< mutex > ul(l);
    while (emptySpotsQ.size() == 0)
        full.wait(ul);
    T *aux = emptySpotsQ.front();
    emptySpotsQ.pop();
    return aux;
}
```

Producers-consumers : buffer I/O exterior to the monitor (3)

```
template<typename T>
T* Monitor<T>::canGet() {
    unique_lock< mutex > ul(l);
    while (itemQ.size() == 0)
        empty.wait(ul);
    T* temp = itemQ.front();
    itemQ.pop();
    return temp;
}
//-----------------------------------

template<typename T>
void Monitor<T>::donePutting(T *x) {
    lock_guard< mutex > lg(l);
    itemQ.push(x);
    empty.notify_one();
}
```

`doneGetting` is also needed.
See book for more.

Producer-consumer functors

```
template<typename T>
void Producer<T>::operator()() {
    while (numProducts.try_acquire()) {
        T item = (*produce)();
        T* aux = mon->canPut();
        *aux = item;
        mon->donePutting(aux);
    }
}
//-------------------------------------
. . .
template<typename T>
void Consumer<T>::operator()() {
    while (numProducts.try_acquire()) {
        T* aux = mon->canGet();
        T item = *aux; // take the item out
        mon->doneGetting(aux);
        (*consume)(item);
    }
}
```

`mon` pointer to shared object must be set prior to running the threads.

Static thread management

- Spawning threads whenever there is a need to off-load/share a workload can be inefficient.
- If this is done frequently, it can be detrimental to performance.
- A better approach is to have a set of threads that could run whatever is "thrown" at them.
- This set of threads that are started only once and just feed from a queue is also known as a **pool of threads.** Advantages:
  - There is no overhead for starting each task, other that the one initially incurred for starting the pool.
  - The number of threads in the pool can be customized to better suit the workload at hand.
  - Thousands of "tasks"/requests can be generated to allow for better load balancing, without being restricted by the O.S.
  - System resource consumption is minimized.
- Disadvantages:
  - Tasks must be packaged in a suitable way that would allow them to be inserted and removed from a queue.
  - There is no - easy - way to enforce task precedence.
  - Having cooperative tasks can be a problem.

"Packaging" tasks

- C++11 provides a `std::packaged_task` class template that can be used to wrap any callable target (e.g. named or unnamed functions) so that it can be called asynchronously.
- The result of the computation is stored in a `std::future` object.
- Example (without an actual thread being used):

```
#include<future>

…

std::packaged_task<int(int,int)> t([](int a, int b)
{return a % b;});

std::future<int> result = t.get_future();

t(1,3);

std::cout << result.get() << endl;
```

A custom thread pool

- Two components :
  - `CustomThreadPool` : buffer for packaged tasks
  - `CustomThread` : thread "feeding" from a `CustomThreadPool` instance

```
template < typename T > class CustomThreadPool;
//*************************************************************
template < typename T > class CustomThread
{
public:
  static CustomThreadPool < T > *tpool;
  void operator () ();
};

//-------------------------------------
template < typename T >
CustomThreadPool < T > *CustomThread < T >::tpool;
//-------------------------------------
template < typename T >
void CustomThread < T >::operator () ()
{
  unique_ptr < packaged_task < T () >> tptr;
  while (tpool->get (tptr))
    {
      packaged_task < T () > task = move (*tptr);
      task ();
    }
}
```

This is the consumer class. User code will serve as the producer.

A custom thread pool, cont.

```
template < typename T >
class CustomThreadPool
{
private:
  condition_variable empty;      // for blocking pool threads if buffer is ↩
     empty
  condition_variable full;       // for blocking calling thread if buffer is ↩
     empty
  mutex l;
  atomic < bool > done;          // flag for termination
  unique_ptr < packaged_task < T () >> *buffer; // pointer to array of ↩
     pointers to objects
  int in = 0, out = 0, count = 0, N, maxThreads;

  unique_ptr < thread > *t;      // threads RAII
public:
  CustomThreadPool (int nt = __NUMTHREADS, int n = __BUFFERSIZE);
  ~CustomThreadPool ();

  bool get (unique_ptr < packaged_task < T () >> &);    // to be called by the↩
      pool threads

  future < T > schedule (unique_ptr < packaged_task < T () > >);  // to be ↩
     called for work request
};
```

Monitor using a shared atomic flag for termination.

A custom thread pool, cont.

```
template < typename T >
future < T > CustomThreadPool < T >::schedule (unique_ptr < packaged_task < T ←
    () > >ct)
{
  unique_lock < mutex > ul (l);
  while (count == N)
    full.wait (ul);
  buffer[in] = move (ct);
  future < T > temp = buffer[in]->get_future ();
  in = (in + 1) % N;
  count++;

  empty.notify_one ();

  return move (temp);
}
```

What producers call...

A custom thread pool, cont.

```
template < typename T >
bool CustomThreadPool < T >::get (unique_ptr < packaged_task < T () >> &↩
     taskptr)
{
  unique_lock < mutex > ul (l);
  while (count == 0 && (done != true))
    empty.wait (ul);

  taskptr = move (buffer[out]);
  out = (out + 1) % N;
  count--;

  full.notify_one ();
  if (done.load () == true && count < 0)          // thread should call get ↩
     again until there are no more pending tasks
    return false;
  else
    return true;
}
```

What consumers call...

Debugging multi-threaded appl.

- Tools & setup:
  - Use a debugger supporting multi-threaded execution (e.g. DDD)
  - Compile your code with debugging information enabled:

    `-g`
  - Do not let the compiler optimize the code (no `-O` switches)
- Application instrumentation:
  - The application should generate some form of log or trace history.
  - Limit or control the number of threads. E.g. one can use a single thread setting to weed-out algorithmic errors.

Debugging multi-threaded appl. (2)

- You can differentiate normal from debugging output by using the standard error.
- The debugging messages can be filtered and redirected to a file. E.g. (works in both Linux and Windows):

```
$ myprog 2> trace.log
```

- Debugging output should be time-stamped.
- Compiler directives can be used to control *when* and *which* of the debugging code is active.

Debugging multi-threaded appl. (3)

```
#include <chrono>

#define DEBUG

//*************************************************
chrono::high_resolution_clock::time_point time0;
mutex l;
double hrclock ()
{
  chrono::high_resolution_clock::time_point t;
  t = chrono::high_resolution_clock::now();
  return chrono::duration<double>(t-time0).count(); //defaults to seconds
}

//*************************************************
void debugMsg (string msg, double timestamp)
{
  l.lock ();
  cerr << timestamp << " " << msg << endl;
  l.unlock ();
}
```

Debugging multi-threaded appl. (4)

```
  void operator()()
  {
    cout << "Thread " << ID << " is running\n";
    for (int j = 0; j < runs; j++)
      {
#ifdef DEBUG
        ostringstream ss;
        ss << "Thread #" << ID << " counter=" << counter;
        debugMsg (ss.str (), hrclock ());
#endif
        this_thread::sleep_for(chrono::duration < int, milli > (rand () % 4 + ↩
    1));
        counter++;
      }
  }
```

```
int main (int argc, char *argv[])
{
#ifdef DEBUG
  time0  = chrono::high_resolution_clock::now();
#endif
. . .
```

A sample run

```
$ ./debugSample 10 100 2> log
Thread Thread Thread Thread Thread Thread Thread
Thread 8 is running
Thread 3 is running
 is running
7 is running
5 is running
Thread 2 is running
0 is running
 is running
 is running
994
$ sort log
. . .
0.0033443 Thread 5 counter=216
0.00335193 Thread 7 counter=218
0.00335193 Thread 8 counter=217      <———
0.00335503 Thread 3 counter=219
```